EDUKRON

300 Practical Oracle SQL Interview Queries

Subqueries | Joins | Window Functions | CTEs | Aggregates | Functions

Five business datasets, 60 distinct SQL patterns, 300 solved interview exercises with expected results.

Dataset setup

Create each table with columns id NUMBER, name VARCHAR2(50), dept VARCHAR2(50), city VARCHAR2(50), amount NUMBER, score NUMBER, status VARCHAR2(20), manager_id NUMBER. The sample rows below are the basis of every displayed result.

employees

```
INSERT INTO employees VALUES (1,'Asha','Engineering','Bengaluru',5000,82,'ACTIVE',NULL);
INSERT INTO employees VALUES (2,'Bala','Engineering','Hyderabad',7000,91,'ACTIVE',1);
INSERT INTO employees VALUES (3,'Chitra','HR','Bengaluru',7000,76,'PENDING',1);
INSERT INTO employees VALUES (4,'Dev','HR','Chennai',3000,65,'ACTIVE',2);
INSERT INTO employees VALUES (5,'Esha','Engineering','Hyderabad',9000,94,'CLOSED',2);
INSERT INTO employees VALUES (6,'Farah','HR','Chennai',4000,88,'ACTIVE',3);
```

bank_accounts

```
INSERT INTO bank_accounts VALUES (1,'Asha','Savings','Bengaluru',5100,82,'ACTIVE',NULL);
INSERT INTO bank_accounts VALUES (2,'Bala','Savings','Hyderabad',7100,91,'ACTIVE',1);
INSERT INTO bank_accounts VALUES (3,'Chitra','Current','Bengaluru',7100,76,'PENDING',1);
INSERT INTO bank_accounts VALUES (4,'Dev','Current','Chennai',3100,65,'ACTIVE',2);
INSERT INTO bank_accounts VALUES (5,'Esha','Savings','Hyderabad',9100,94,'CLOSED',2);
INSERT INTO bank_accounts VALUES (6,'Farah','Current','Chennai',4100,88,'ACTIVE',3);
```

retail_orders

```
INSERT INTO retail_orders VALUES (1,'Asha','Online','Bengaluru',5200,82,'ACTIVE',NULL);
INSERT INTO retail_orders VALUES (2,'Bala','Online','Hyderabad',7200,91,'ACTIVE',1);
INSERT INTO retail_orders VALUES (3,'Chitra','Store','Bengaluru',7200,76,'PENDING',1);
INSERT INTO retail_orders VALUES (4,'Dev','Store','Chennai',3200,65,'ACTIVE',2);
INSERT INTO retail_orders VALUES (5,'Esha','Online','Hyderabad',9200,94,'CLOSED',2);
INSERT INTO retail_orders VALUES (6,'Farah','Store','Chennai',4200,88,'ACTIVE',3);
```

insurance_claims

```
INSERT INTO insurance_claims VALUES (1,'Asha','Health','Bengaluru',5300,82,'ACTIVE',NULL);
INSERT INTO insurance_claims VALUES (2,'Bala','Health','Hyderabad',7300,91,'ACTIVE',1);
INSERT INTO insurance_claims VALUES (3,'Chitra','Motor','Bengaluru',7300,76,'PENDING',1);
INSERT INTO insurance_claims VALUES (4,'Dev','Motor','Chennai',3300,65,'ACTIVE',2);
INSERT INTO insurance_claims VALUES (5,'Esha','Health','Hyderabad',9300,94,'CLOSED',2);
INSERT INTO insurance_claims VALUES (6,'Farah','Motor','Chennai',4300,88,'ACTIVE',3);
```

student_fees

```
INSERT INTO student_fees VALUES (1,'Asha','Data Engineering','Bengaluru',5400,82,'ACTIVE',NULL);
INSERT INTO student_fees VALUES (2,'Bala','Data Engineering','Hyderabad',7400,91,'ACTIVE',1);
INSERT INTO student_fees VALUES (3,'Chitra','DevOps','Bengaluru',7400,76,'PENDING',1);
INSERT INTO student_fees VALUES (4,'Dev','DevOps','Chennai',3400,65,'ACTIVE',2);
INSERT INTO student_fees VALUES (5,'Esha','Data Engineering','Hyderabad',9400,94,'CLOSED',2);
INSERT INTO student_fees VALUES (6,'Farah','DevOps','Chennai',4400,88,'ACTIVE',3);
```

Business case 1: Employees

Q1. Active values above 5000 — employees

```
SELECT name,amount FROM employees WHERE status='ACTIVE' AND amount>5000 ORDER BY id
```

Expected output:

```
Bala | 7000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q2. Values within a range — employees

```
SELECT name,amount FROM employees WHERE amount BETWEEN 4000 AND 7000 ORDER BY amount,name
```

Expected output:

```
Farah | 4000
Asha | 5000
Bala | 7000
Chitra | 7000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q3. Cities using IN — employees

```
SELECT name,city FROM employees WHERE city IN ('Chennai','Hyderabad') ORDER BY name
```

Expected output:

```
Bala | Hyderabad
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q4. Names starting with A — employees

```
SELECT name FROM employees WHERE name LIKE 'A%'
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q5. Records with missing managers — employees

```
SELECT name FROM employees WHERE manager_id IS NULL
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q6. Records with valid manager IDs — employees

```
SELECT name,manager_id FROM employees WHERE manager_id IS NOT NULL ORDER BY id
```

Expected output:

```
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q7. Exclude closed records — employees

```
SELECT name,status FROM employees WHERE status<>'CLOSED' ORDER BY id
```

Expected output:

```
Asha | ACTIVE
Bala | ACTIVE
Chitra | PENDING
Dev | ACTIVE
Farah | ACTIVE
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q8. First category only — employees

```
SELECT name FROM employees WHERE dept='Engineering' ORDER BY name
```

Expected output:

```
Asha
Bala
Esha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q9. Descending values and stable ties — employees

```
SELECT name,amount FROM employees ORDER BY amount DESC,name
```

Expected output:

```
Esha | 9000
Bala | 7000
Chitra | 7000
Asha | 5000
Farah | 4000
Dev | 3000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q10. Scores above 85 — employees

```
SELECT name,score FROM employees WHERE score>85 ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q11. Count records by status — employees

```
SELECT status,COUNT(*) AS n FROM employees GROUP BY status ORDER BY status
```

Expected output:

```
ACTIVE | 4
CLOSED | 1
PENDING | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q12. Sum amount by category — employees

```
SELECT dept,SUM(amount) AS total FROM employees GROUP BY dept ORDER BY dept
```

Expected output:

```
Engineering | 21000
HR | 14000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q13. Average amount by city — employees

```
SELECT city,ROUND(AVG(amount),2) AS avg_amount FROM employees GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 6000.0
Chennai | 3500.0
Hyderabad | 8000.0
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q14. Maximum amount by category — employees

```
SELECT dept,MAX(amount) AS maximum FROM employees GROUP BY dept ORDER BY dept
```

Expected output:

```
Engineering | 9000
HR | 7000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q15. Minimum amount by city — employees

```
SELECT city,MIN(amount) AS minimum FROM employees GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 5000
Chennai | 3000
Hyderabad | 7000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q16. Distinct active cities — employees

```
SELECT COUNT(DISTINCT city) AS cities FROM employees WHERE status='ACTIVE'
```

Expected output:

```
3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q17. Categories with three records — employees

```
SELECT dept,COUNT(*) AS n FROM employees GROUP BY dept HAVING COUNT(*)>=3 ORDER BY dept
```

Expected output:

```
Engineering | 3
HR | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q18. Conditional active sum — employees

```
SELECT SUM(CASE WHEN status='ACTIVE' THEN amount ELSE 0 END) AS active_total FROM employees
```

Expected output:

```
19000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q19. Cities having multiple records — employees

```
SELECT city,COUNT(*) AS n FROM employees GROUP BY city HAVING COUNT(*)>1 ORDER BY city
```

Expected output:

```
Bengaluru | 2
Chennai | 2
Hyderabad | 2
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q20. Grand total and average — employees

```
SELECT SUM(amount) AS total,ROUND(AVG(amount),2) AS average FROM employees
```

Expected output:

```
35000 | 5833.33
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q21. Above overall average — employees

```
SELECT name,amount FROM employees WHERE amount>(SELECT AVG(amount) FROM employees) ORDER BY
amount DESC
```

Expected output:

```
Esha | 9000
Bala | 7000
Chitra | 7000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q22. Highest amount including ties — employees

```
SELECT name,amount FROM employees WHERE amount=(SELECT MAX(amount) FROM employees)
```

Expected output:

```
Esha | 9000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q23. Second distinct highest amount — employees

```
SELECT name,amount FROM employees WHERE amount=(SELECT MAX(amount) FROM employees WHERE
amount<(SELECT MAX(amount) FROM employees)) ORDER BY name
```

Expected output:

```
Bala | 7000
Chitra | 7000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q24. Above own category average — employees

```
SELECT x.name,x.amount FROM employees x WHERE x.amount>(SELECT AVG(y.amount) FROM employees y
WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Chitra | 7000
Esha | 9000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q25. Records from repeated cities — employees

```
SELECT name,city FROM employees WHERE city IN (SELECT city FROM employees GROUP BY city HAVING
COUNT(*)>1) ORDER BY name
```

Expected output:

```
Asha | Bengaluru
Bala | Hyderabad
Chitra | Bengaluru
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q26. Records with existing manager — employees

```
SELECT x.name FROM employees x WHERE EXISTS(SELECT 1 FROM employees m WHERE m.id=x.manager_id)
ORDER BY x.name
```

Expected output:

```
Bala
Chitra
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q27. Records managing nobody — employees

```
SELECT x.name FROM employees x WHERE NOT EXISTS(SELECT 1 FROM employees y WHERE
y.manager_id=x.id) ORDER BY x.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q28. Category-wise minimum records — employees

```
SELECT x.name,x.amount FROM employees x WHERE x.amount=(SELECT MIN(y.amount) FROM employees y
WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Asha | 5000
Dev | 3000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q29. First category above second category maximum — employees

```
SELECT name,amount FROM employees WHERE dept='Engineering' AND amount>(SELECT MAX(amount) FROM
employees WHERE dept='HR')
```

Expected output:

```
Esha | 9000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q30. First category below second category maximum — employees

```
SELECT name,amount FROM employees WHERE dept='Engineering' AND amount<(SELECT MAX(amount) FROM
employees WHERE dept='HR')
```

Expected output:

```
Asha | 5000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q31. Manager names using self join — employees

```
SELECT e.name AS member,m.name AS manager FROM employees e LEFT JOIN employees m ON
m.id=e.manager_id ORDER BY e.id
```

Expected output:

```
Asha | NULL
Bala | Asha
Chitra | Asha
Dev | Bala
Esha | Bala
Farah | Chitra
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q32. Direct reports per manager — employees

```
SELECT m.name,COUNT(e.id) AS reports FROM employees m JOIN employees e ON e.manager_id=m.id
GROUP BY m.id,m.name ORDER BY m.id
```

Expected output:

```
Asha | 2
Bala | 2
Chitra | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q33. Managers in same city — employees

```
SELECT e.name,m.name AS manager FROM employees e JOIN employees m ON m.id=e.manager_id AND
m.city=e.city ORDER BY e.name
```

Expected output:

```
Chitra | Asha
Esha | Bala
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q34. No direct reports using anti join — employees

```
SELECT m.name FROM employees m LEFT JOIN employees e ON e.manager_id=m.id WHERE e.id IS NULL
ORDER BY m.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q35. Unique peer pairs — employees

```
SELECT a.name,b.name AS peer FROM employees a JOIN employees b ON a.dept=b.dept AND a.id<b.id
ORDER BY a.name,b.name
```

Expected output:

```
Asha | Bala
Asha | Esha
Bala | Esha
Chitra | Dev
Chitra | Farah
Dev | Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q36. Rank by amount — employees

```
SELECT name,amount,RANK() OVER(ORDER BY amount DESC) AS rnk FROM employees ORDER BY rnk,name
```

Expected output:

```
Esha | 9000 | 1
Bala | 7000 | 2
Chitra | 7000 | 2
Asha | 5000 | 4
Farah | 4000 | 5
Dev | 3000 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q37. Dense rank by category — employees

```
SELECT name,dept,DENSE_RANK() OVER(PARTITION BY dept ORDER BY amount DESC) AS rnk FROM
employees ORDER BY dept,rnk,name
```

Expected output:

```
Esha | Engineering | 1
Bala | Engineering | 2
Asha | Engineering | 3
Chitra | HR | 1
Farah | HR | 2
Dev | HR | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q38. Top two per category — employees

```
SELECT name,dept,amount FROM (SELECT name,dept,amount,ROW_NUMBER() OVER(PARTITION BY dept
ORDER BY amount DESC,id) rn FROM employees) WHERE rn<=2 ORDER BY dept,amount DESC
```

Expected output:

```
Esha | Engineering | 9000
Bala | Engineering | 7000
Chitra | HR | 7000
Farah | HR | 4000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q39. Running total — employees

```
SELECT name,amount,SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS running_total FROM employees ORDER BY id
```

Expected output:

```
Asha | 5000 | 5000
Bala | 7000 | 12000
Chitra | 7000 | 19000
Dev | 3000 | 22000
Esha | 9000 | 31000
Farah | 4000 | 35000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q40. Percentage of total — employees

```
SELECT name,ROUND(100.0*amount/SUM(amount) OVER(),2) AS pct FROM employees ORDER BY id
```

Expected output:

```
Asha | 14.29
Bala | 20.0
Chitra | 20.0
Dev | 8.57
Esha | 25.71
Farah | 11.43
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q41. Previous amount with LAG — employees

```
SELECT name,amount,LAG(amount) OVER(ORDER BY id) AS previous FROM employees ORDER BY id
```

Expected output:

```
Asha | 5000 | NULL
Bala | 7000 | 5000
Chitra | 7000 | 7000
Dev | 3000 | 7000
Esha | 9000 | 3000
```

```
Farah | 4000 | 9000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q42. Next amount with LEAD — employees

```
SELECT name,amount,LEAD(amount) OVER(ORDER BY id) AS next_amount FROM employees ORDER BY id
```

Expected output:

```
Asha | 5000 | 7000
Bala | 7000 | 7000
Chitra | 7000 | 3000
Dev | 3000 | 9000
Esha | 9000 | 4000
Farah | 4000 | NULL
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q43. Category total on every row — employees

```
SELECT name,dept,SUM(amount) OVER(PARTITION BY dept) AS dept_total FROM employees ORDER BY id
```

Expected output:

```
Asha | Engineering | 21000
Bala | Engineering | 21000
Chitra | HR | 14000
Dev | HR | 14000
Esha | Engineering | 21000
Farah | HR | 14000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q44. Row number by score — employees

```
SELECT name,score,ROW_NUMBER() OVER(ORDER BY score DESC,id) AS position FROM employees ORDER
BY position
```

Expected output:

```
Esha | 94 | 1
Bala | 91 | 2
Farah | 88 | 3
Asha | 82 | 4
Chitra | 76 | 5
Dev | 65 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q45. Difference from category average — employees

```
SELECT name,ROUND(amount-AVG(amount) OVER(PARTITION BY dept),2) AS delta FROM employees ORDER
BY id
```

Expected output:

```
Asha | -2000.0
Bala | 0.0
Chitra | 2333.33
Dev | -1666.67
Esha | 2000.0
Farah | -666.67
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q46. Uppercase names — employees

```
SELECT name,UPPER(name) AS normalized FROM employees ORDER BY id
```

Expected output:

```
Asha | ASHA
Bala | BALA
Chitra | CHITRA
Dev | DEV
Esha | ESHA
Farah | FARAH
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q47. First three name characters — employees

```
SELECT name,SUBSTR(name,1,3) AS prefix FROM employees ORDER BY id
```

Expected output:

```
Asha | Ash
Bala | Bal
Chitra | Chi
Dev | Dev
Esha | Esh
Farah | Far
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q48. Name length — employees

```
SELECT name,LENGTH(name) AS chars FROM employees ORDER BY id
```

Expected output:

```
Asha | 4
Bala | 4
Chitra | 6
Dev | 3
Esha | 4
Farah | 5
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q49. Mask lowercase a — employees

```
SELECT name,REPLACE(name,'a','*') AS masked FROM employees ORDER BY id
```

Expected output:

```
Asha | Ash*
Bala | B*l*
Chitra | Chitr*
Dev | Dev
Esha | Esh*
Farah | F*r*h
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q50. Amount classification using CASE — employees

```
SELECT name,CASE WHEN amount>=8000 THEN 'HIGH' WHEN amount>=5000 THEN 'MEDIUM' ELSE 'LOW' END
AS band FROM employees ORDER BY id
```

Expected output:

```
Asha | MEDIUM
Bala | MEDIUM
Chitra | MEDIUM
Dev | LOW
Esha | HIGH
Farah | LOW
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q51. Replace missing manager with zero — employees

```
SELECT name,COALESCE(manager_id,0) AS manager_key FROM employees ORDER BY id
```

Expected output:

```
Asha | 0
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q52. Absolute deviation from 6000 — employees

```
SELECT name,ABS(amount-6000) AS deviation FROM employees ORDER BY id
```

Expected output:

```
Asha | 1000
Bala | 1000
Chitra | 1000
Dev | 3000
Esha | 3000
Farah | 2000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q53. Score remainder — employees

```
SELECT name,MOD(score,10) AS remainder FROM employees ORDER BY id
```

Expected output:

```
Asha | 2
Bala | 1
Chitra | 6
Dev | 5
Esha | 4
Farah | 8
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q54. Concatenate category and city — employees

```
SELECT name,dept||' / '||city AS label FROM employees ORDER BY id
```

Expected output:

```
Asha | Engineering / Bengaluru
Bala | Engineering / Hyderabad
Chitra | HR / Bengaluru
Dev | HR / Chennai
Esha | Engineering / Hyderabad
Farah | HR / Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q55. Average score rounded — employees

```
SELECT dept,ROUND(AVG(score),1) AS avg_score FROM employees GROUP BY dept ORDER BY dept
```

Expected output:

```
Engineering | 89.0
HR | 76.3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q56. CTE filtering category totals — employees

```
WITH totals AS(SELECT dept,SUM(amount) total FROM employees GROUP BY dept) SELECT dept,total
FROM totals WHERE total>15000 ORDER BY dept
```

Expected output:

```
Engineering | 21000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q57. CTE for high scores — employees

```
WITH qualified AS(SELECT name,score FROM employees WHERE score>=85) SELECT name,score FROM
qualified ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q58. CTE for highest per city — employees

```
WITH ranked AS(SELECT name,city,amount,ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount
DESC,id) rn FROM employees) SELECT name,city,amount FROM ranked WHERE rn=1 ORDER BY city
```

Expected output:

```
Chitra | Bengaluru | 7000
Farah | Chennai | 4000
Esha | Hyderabad | 9000
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q59. UNION active and pending cities — employees

```
SELECT city FROM employees WHERE status='ACTIVE' UNION SELECT city FROM employees WHERE
status='PENDING' ORDER BY city
```

Expected output:

```
Bengaluru
Chennai
Hyderabad
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q60. INTERSECT cities across categories — employees

```
SELECT city FROM employees WHERE dept='Engineering' INTERSECT SELECT city FROM employees WHERE
dept='HR' ORDER BY city
```

Expected output:

```
Bengaluru
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Business case 2: Bank Accounts

Q61. Active values above 5000 — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE status='ACTIVE' AND amount>5000 ORDER BY id
```

Expected output:

```
Asha | 5100
Bala | 7100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q62. Values within a range — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE amount BETWEEN 4000 AND 7000 ORDER BY amount,name
```

Expected output:

```
Farah | 4100
Asha | 5100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q63. Cities using IN — bank accounts

```
SELECT name,city FROM bank_accounts WHERE city IN ('Chennai','Hyderabad') ORDER BY name
```

Expected output:

```
Bala | Hyderabad
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q64. Names starting with A — bank accounts

```
SELECT name FROM bank_accounts WHERE name LIKE 'A%'
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q65. Records with missing managers — bank accounts

```
SELECT name FROM bank_accounts WHERE manager_id IS NULL
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q66. Records with valid manager IDs — bank accounts

```
SELECT name,manager_id FROM bank_accounts WHERE manager_id IS NOT NULL ORDER BY id
```

Expected output:

```
Bala | 1
```

```
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q67. Exclude closed records — bank accounts

```
SELECT name,status FROM bank_accounts WHERE status<>'CLOSED' ORDER BY id
```

Expected output:

```
Asha | ACTIVE
Bala | ACTIVE
Chitra | PENDING
Dev | ACTIVE
Farah | ACTIVE
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q68. First category only — bank accounts

```
SELECT name FROM bank_accounts WHERE dept='Savings' ORDER BY name
```

Expected output:

```
Asha
Bala
Esha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q69. Descending values and stable ties — bank accounts

```
SELECT name,amount FROM bank_accounts ORDER BY amount DESC,name
```

Expected output:

```
Esha | 9100
Bala | 7100
Chitra | 7100
Asha | 5100
Farah | 4100
Dev | 3100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q70. Scores above 85 — bank accounts

```
SELECT name,score FROM bank_accounts WHERE score>85 ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q71. Count records by status — bank accounts

```
SELECT status,COUNT(*) AS n FROM bank_accounts GROUP BY status ORDER BY status
```

Expected output:

```
ACTIVE | 4
CLOSED | 1
PENDING | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q72. Sum amount by category — bank accounts

```
SELECT dept,SUM(amount) AS total FROM bank_accounts GROUP BY dept ORDER BY dept
```

Expected output:

```
Current | 14300
Savings | 21300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q73. Average amount by city — bank accounts

```
SELECT city,ROUND(AVG(amount),2) AS avg_amount FROM bank_accounts GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 6100.0
Chennai | 3600.0
Hyderabad | 8100.0
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q74. Maximum amount by category — bank accounts

```
SELECT dept,MAX(amount) AS maximum FROM bank_accounts GROUP BY dept ORDER BY dept
```

Expected output:

```
Current | 7100
Savings | 9100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q75. Minimum amount by city — bank accounts

```
SELECT city,MIN(amount) AS minimum FROM bank_accounts GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 5100
Chennai | 3100
Hyderabad | 7100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q76. Distinct active cities — bank accounts

```
SELECT COUNT(DISTINCT city) AS cities FROM bank_accounts WHERE status='ACTIVE'
```

Expected output:

```
3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q77. Categories with three records — bank accounts

```
SELECT dept,COUNT(*) AS n FROM bank_accounts GROUP BY dept HAVING COUNT(*)>=3 ORDER BY dept
```

Expected output:

```
Current | 3
Savings | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q78. Conditional active sum — bank accounts

```
SELECT SUM(CASE WHEN status='ACTIVE' THEN amount ELSE 0 END) AS active_total FROM bank_accounts
```

Expected output:

```
19400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q79. Cities having multiple records — bank accounts

```
SELECT city,COUNT(*) AS n FROM bank_accounts GROUP BY city HAVING COUNT(*)>1 ORDER BY city
```

Expected output:

```
Bengaluru | 2
Chennai | 2
Hyderabad | 2
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q80. Grand total and average — bank accounts

```
SELECT SUM(amount) AS total,ROUND(AVG(amount),2) AS average FROM bank_accounts
```

Expected output:

```
35600 | 5933.33
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q81. Above overall average — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE amount>(SELECT AVG(amount) FROM bank_accounts)
ORDER BY amount DESC
```

Expected output:

```
Esha | 9100
Bala | 7100
Chitra | 7100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q82. Highest amount including ties — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE amount=(SELECT MAX(amount) FROM bank_accounts)
```

Expected output:

```
Esha | 9100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q83. Second distinct highest amount — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE amount=(SELECT MAX(amount) FROM bank_accounts
WHERE amount<(SELECT MAX(amount) FROM bank_accounts)) ORDER BY name
```

Expected output:

```
Bala | 7100
Chitra | 7100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q84. Above own category average — bank accounts

```
SELECT x.name,x.amount FROM bank_accounts x WHERE x.amount>(SELECT AVG(y.amount) FROM
bank_accounts y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Chitra | 7100
Esha | 9100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q85. Records from repeated cities — bank accounts

```
SELECT name,city FROM bank_accounts WHERE city IN (SELECT city FROM bank_accounts GROUP BY
city HAVING COUNT(*)>1) ORDER BY name
```

Expected output:

```
Asha | Bengaluru
Bala | Hyderabad
Chitra | Bengaluru
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q86. Records with existing manager — bank accounts

```
SELECT x.name FROM bank_accounts x WHERE EXISTS(SELECT 1 FROM bank_accounts m WHERE
m.id=x.manager_id) ORDER BY x.name
```

Expected output:

```
Bala
Chitra
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q87. Records managing nobody — bank accounts

```
SELECT x.name FROM bank_accounts x WHERE NOT EXISTS(SELECT 1 FROM bank_accounts y WHERE
y.manager_id=x.id) ORDER BY x.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q88. Category-wise minimum records — bank accounts

```
SELECT x.name,x.amount FROM bank_accounts x WHERE x.amount=(SELECT MIN(y.amount) FROM
bank_accounts y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Asha | 5100
Dev | 3100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q89. First category above second category maximum — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE dept='Savings' AND amount>(SELECT MAX(amount) FROM
bank_accounts WHERE dept='Current')
```

Expected output:

```
Esha | 9100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q90. First category below second category maximum — bank accounts

```
SELECT name,amount FROM bank_accounts WHERE dept='Savings' AND amount<(SELECT MAX(amount) FROM
bank_accounts WHERE dept='Current')
```

Expected output:

```
Asha | 5100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q91. Manager names using self join — bank accounts

```
SELECT e.name AS member,m.name AS manager FROM bank_accounts e LEFT JOIN bank_accounts m ON
m.id=e.manager_id ORDER BY e.id
```

Expected output:

```
Asha | NULL
Bala | Asha
Chitra | Asha
Dev | Bala
Esha | Bala
Farah | Chitra
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q92. Direct reports per manager — bank accounts

```
SELECT m.name,COUNT(e.id) AS reports FROM bank_accounts m JOIN bank_accounts e ON
e.manager_id=m.id GROUP BY m.id,m.name ORDER BY m.id
```

Expected output:

```
Asha | 2
Bala | 2
Chitra | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q93. Managers in same city — bank accounts

```
SELECT e.name,m.name AS manager FROM bank_accounts e JOIN bank_accounts m ON m.id=e.manager_id
AND m.city=e.city ORDER BY e.name
```

Expected output:

```
Chitra | Asha
Esha | Bala
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q94. No direct reports using anti join — bank accounts

```
SELECT m.name FROM bank_accounts m LEFT JOIN bank_accounts e ON e.manager_id=m.id WHERE e.id
IS NULL ORDER BY m.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q95. Unique peer pairs — bank accounts

```
SELECT a.name,b.name AS peer FROM bank_accounts a JOIN bank_accounts b ON a.dept=b.dept AND
a.id<b.id ORDER BY a.name,b.name
```

Expected output:

```
Asha | Bala
Asha | Esha
Bala | Esha
Chitra | Dev
Chitra | Farah
Dev | Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q96. Rank by amount — bank accounts

```
SELECT name,amount,RANK() OVER(ORDER BY amount DESC) AS rnk FROM bank_accounts ORDER BY rnk,
name
```

Expected output:

```
Esha | 9100 | 1
Bala | 7100 | 2
Chitra | 7100 | 2
Asha | 5100 | 4
Farah | 4100 | 5
Dev | 3100 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q97. Dense rank by category — bank accounts

```
SELECT name,dept,DENSE_RANK() OVER(PARTITION BY dept ORDER BY amount DESC) AS rnk FROM
bank_accounts ORDER BY dept,rnk,name
```

Expected output:

```
Chitra | Current | 1
Farah | Current | 2
Dev | Current | 3
Esha | Savings | 1
Bala | Savings | 2
Asha | Savings | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q98. Top two per category — bank accounts

```
SELECT name,dept,amount FROM (SELECT name,dept,amount,ROW_NUMBER() OVER(PARTITION BY dept
ORDER BY amount DESC,id) rn FROM bank_accounts) WHERE rn<=2 ORDER BY dept,amount DESC
```

Expected output:

```
Chitra | Current | 7100
Farah | Current | 4100
Esha | Savings | 9100
Bala | Savings | 7100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q99. Running total — bank accounts

```
SELECT name,amount,SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS running_total FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 5100 | 5100
Bala | 7100 | 12200
Chitra | 7100 | 19300
Dev | 3100 | 22400
Esha | 9100 | 31500
Farah | 4100 | 35600
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q100. Percentage of total — bank accounts

```
SELECT name,ROUND(100.0*amount/SUM(amount) OVER(),2) AS pct FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 14.33
Bala | 19.94
Chitra | 19.94
Dev | 8.71
Esha | 25.56
Farah | 11.52
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q101. Previous amount with LAG — bank accounts

```
SELECT name,amount,LAG(amount) OVER(ORDER BY id) AS previous FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 5100 | NULL
Bala | 7100 | 5100
Chitra | 7100 | 7100
Dev | 3100 | 7100
Esha | 9100 | 3100
```

```
Farah | 4100 | 9100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q102. Next amount with LEAD — bank accounts

```
SELECT name,amount,LEAD(amount) OVER(ORDER BY id) AS next_amount FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 5100 | 7100
Bala | 7100 | 7100
Chitra | 7100 | 3100
Dev | 3100 | 9100
Esha | 9100 | 4100
Farah | 4100 | NULL
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q103. Category total on every row — bank accounts

```
SELECT name,dept,SUM(amount) OVER(PARTITION BY dept) AS dept_total FROM bank_accounts ORDER BY
id
```

Expected output:

```
Asha | Savings | 21300
Bala | Savings | 21300
Chitra | Current | 14300
Dev | Current | 14300
Esha | Savings | 21300
Farah | Current | 14300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q104. Row number by score — bank accounts

```
SELECT name,score,ROW_NUMBER() OVER(ORDER BY score DESC,id) AS position FROM bank_accounts
ORDER BY position
```

Expected output:

```
Esha | 94 | 1
Bala | 91 | 2
Farah | 88 | 3
Asha | 82 | 4
Chitra | 76 | 5
Dev | 65 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q105. Difference from category average — bank accounts

```
SELECT name,ROUND(amount-AVG(amount) OVER(PARTITION BY dept),2) AS delta FROM bank_accounts
ORDER BY id
```

Expected output:

```
Asha | -2000.0
Bala | 0.0
Chitra | 2333.33
Dev | -1666.67
Esha | 2000.0
Farah | -666.67
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q106. Uppercase names — bank accounts

```
SELECT name,UPPER(name) AS normalized FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | ASHA
Bala | BALA
Chitra | CHITRA
Dev | DEV
Esha | ESHA
Farah | FARAH
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q107. First three name characters — bank accounts

```
SELECT name,SUBSTR(name,1,3) AS prefix FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | Ash
Bala | Bal
Chitra | Chi
Dev | Dev
Esha | Esh
Farah | Far
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q108. Name length — bank accounts

```
SELECT name,LENGTH(name) AS chars FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 4
Bala | 4
Chitra | 6
Dev | 3
Esha | 4
Farah | 5
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q109. Mask lowercase a — bank accounts

```
SELECT name,REPLACE(name,'a','*') AS masked FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | Ash*
Bala | B*l*
Chitra | Chitr*
Dev | Dev
Esha | Esh*
Farah | F*r*h
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q110. Amount classification using CASE — bank accounts

```
SELECT name,CASE WHEN amount>=8000 THEN 'HIGH' WHEN amount>=5000 THEN 'MEDIUM' ELSE 'LOW' END
AS band FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | MEDIUM
Bala | MEDIUM
Chitra | MEDIUM
Dev | LOW
Esha | HIGH
Farah | LOW
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q111. Replace missing manager with zero — bank accounts

```
SELECT name,COALESCE(manager_id,0) AS manager_key FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 0
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q112. Absolute deviation from 6000 — bank accounts

```
SELECT name,ABS(amount-6000) AS deviation FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 900
Bala | 1100
Chitra | 1100
Dev | 2900
Esha | 3100
Farah | 1900
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q113. Score remainder — bank accounts

```
SELECT name,MOD(score,10) AS remainder FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | 2
Bala | 1
Chitra | 6
Dev | 5
Esha | 4
Farah | 8
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q114. Concatenate category and city — bank accounts

```
SELECT name,dept||' / '||city AS label FROM bank_accounts ORDER BY id
```

Expected output:

```
Asha | Savings / Bengaluru
Bala | Savings / Hyderabad
Chitra | Current / Bengaluru
Dev | Current / Chennai
Esha | Savings / Hyderabad
```

```
Farah | Current / Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q115. Average score rounded — bank accounts

```
SELECT dept,ROUND(AVG(score),1) AS avg_score FROM bank_accounts GROUP BY dept ORDER BY dept
```

Expected output:

```
Current | 76.3
Savings | 89.0
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q116. CTE filtering category totals — bank accounts

```
WITH totals AS(SELECT dept,SUM(amount) total FROM bank_accounts GROUP BY dept) SELECT dept,
total FROM totals WHERE total>15000 ORDER BY dept
```

Expected output:

```
Savings | 21300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q117. CTE for high scores — bank accounts

```
WITH qualified AS(SELECT name,score FROM bank_accounts WHERE score>=85) SELECT name,score FROM
qualified ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q118. CTE for highest per city — bank accounts

```
WITH ranked AS(SELECT name,city,amount,ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount
DESC,id) rn FROM bank_accounts) SELECT name,city,amount FROM ranked WHERE rn=1 ORDER BY city
```

Expected output:

```
Chitra | Bengaluru | 7100
Farah | Chennai | 4100
Esha | Hyderabad | 9100
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q119. UNION active and pending cities — bank accounts

```
SELECT city FROM bank_accounts WHERE status='ACTIVE' UNION SELECT city FROM bank_accounts
WHERE status='PENDING' ORDER BY city
```

Expected output:

```
Bengaluru
Chennai
Hyderabad
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q120. INTERSECT cities across categories — bank accounts

```
SELECT city FROM bank_accounts WHERE dept='Savings' INTERSECT SELECT city FROM bank_accounts
WHERE dept='Current' ORDER BY city
```

Expected output:

```
Bengaluru
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Business case 3: Retail Orders

Q121. Active values above 5000 — retail orders

```
SELECT name,amount FROM retail_orders WHERE status='ACTIVE' AND amount>5000 ORDER BY id
```

Expected output:

```
Asha | 5200
Bala | 7200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q122. Values within a range — retail orders

```
SELECT name,amount FROM retail_orders WHERE amount BETWEEN 4000 AND 7000 ORDER BY amount,name
```

Expected output:

```
Farah | 4200
Asha | 5200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q123. Cities using IN — retail orders

```
SELECT name,city FROM retail_orders WHERE city IN ('Chennai','Hyderabad') ORDER BY name
```

Expected output:

```
Bala | Hyderabad
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q124. Names starting with A — retail orders

```
SELECT name FROM retail_orders WHERE name LIKE 'A%'
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q125. Records with missing managers — retail orders

```
SELECT name FROM retail_orders WHERE manager_id IS NULL
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q126. Records with valid manager IDs — retail orders

```
SELECT name,manager_id FROM retail_orders WHERE manager_id IS NOT NULL ORDER BY id
```

Expected output:

```
Bala | 1
```

```
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q127. Exclude closed records — retail orders

```
SELECT name,status FROM retail_orders WHERE status<>'CLOSED' ORDER BY id
```

Expected output:

```
Asha | ACTIVE
Bala | ACTIVE
Chitra | PENDING
Dev | ACTIVE
Farah | ACTIVE
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q128. First category only — retail orders

```
SELECT name FROM retail_orders WHERE dept='Online' ORDER BY name
```

Expected output:

```
Asha
Bala
Esha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q129. Descending values and stable ties — retail orders

```
SELECT name,amount FROM retail_orders ORDER BY amount DESC,name
```

Expected output:

```
Esha | 9200
Bala | 7200
Chitra | 7200
Asha | 5200
Farah | 4200
Dev | 3200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q130. Scores above 85 — retail orders

```
SELECT name,score FROM retail_orders WHERE score>85 ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q131. Count records by status — retail orders

```
SELECT status,COUNT(*) AS n FROM retail_orders GROUP BY status ORDER BY status
```

Expected output:

```
ACTIVE | 4
CLOSED | 1
PENDING | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q132. Sum amount by category — retail orders

```
SELECT dept,SUM(amount) AS total FROM retail_orders GROUP BY dept ORDER BY dept
```

Expected output:

```
Online | 21600
Store | 14600
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q133. Average amount by city — retail orders

```
SELECT city,ROUND(AVG(amount),2) AS avg_amount FROM retail_orders GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 6200.0
Chennai | 3700.0
Hyderabad | 8200.0
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q134. Maximum amount by category — retail orders

```
SELECT dept,MAX(amount) AS maximum FROM retail_orders GROUP BY dept ORDER BY dept
```

Expected output:

```
Online | 9200
Store | 7200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q135. Minimum amount by city — retail orders

```
SELECT city,MIN(amount) AS minimum FROM retail_orders GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 5200
Chennai | 3200
Hyderabad | 7200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q136. Distinct active cities — retail orders

```
SELECT COUNT(DISTINCT city) AS cities FROM retail_orders WHERE status='ACTIVE'
```

Expected output:

```
3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q137. Categories with three records — retail orders

```
SELECT dept,COUNT(*) AS n FROM retail_orders GROUP BY dept HAVING COUNT(*)>=3 ORDER BY dept
```

Expected output:

```
Online | 3
Store | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q138. Conditional active sum — retail orders

```
SELECT SUM(CASE WHEN status='ACTIVE' THEN amount ELSE 0 END) AS active_total FROM retail_orders
```

Expected output:

```
19800
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q139. Cities having multiple records — retail orders

```
SELECT city,COUNT(*) AS n FROM retail_orders GROUP BY city HAVING COUNT(*)>1 ORDER BY city
```

Expected output:

```
Bengaluru | 2
Chennai | 2
Hyderabad | 2
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q140. Grand total and average — retail orders

```
SELECT SUM(amount) AS total,ROUND(AVG(amount),2) AS average FROM retail_orders
```

Expected output:

```
36200 | 6033.33
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q141. Above overall average — retail orders

```
SELECT name,amount FROM retail_orders WHERE amount>(SELECT AVG(amount) FROM retail_orders)
ORDER BY amount DESC
```

Expected output:

```
Esha | 9200
Bala | 7200
Chitra | 7200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q142. Highest amount including ties — retail orders

```
SELECT name,amount FROM retail_orders WHERE amount=(SELECT MAX(amount) FROM retail_orders)
```

Expected output:

```
Esha | 9200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q143. Second distinct highest amount — retail orders

```
SELECT name,amount FROM retail_orders WHERE amount=(SELECT MAX(amount) FROM retail_orders
WHERE amount<(SELECT MAX(amount) FROM retail_orders)) ORDER BY name
```

Expected output:

```
Bala | 7200
Chitra | 7200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q144. Above own category average — retail orders

```
SELECT x.name,x.amount FROM retail_orders x WHERE x.amount>(SELECT AVG(y.amount) FROM
retail_orders y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Chitra | 7200
Esha | 9200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q145. Records from repeated cities — retail orders

```
SELECT name,city FROM retail_orders WHERE city IN (SELECT city FROM retail_orders GROUP BY
city HAVING COUNT(*)>1) ORDER BY name
```

Expected output:

```
Asha | Bengaluru
Bala | Hyderabad
Chitra | Bengaluru
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q146. Records with existing manager — retail orders

```
SELECT x.name FROM retail_orders x WHERE EXISTS(SELECT 1 FROM retail_orders m WHERE
m.id=x.manager_id) ORDER BY x.name
```

Expected output:

```
Bala
Chitra
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q147. Records managing nobody — retail orders

```
SELECT x.name FROM retail_orders x WHERE NOT EXISTS(SELECT 1 FROM retail_orders y WHERE
y.manager_id=x.id) ORDER BY x.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q148. Category-wise minimum records — retail orders

```
SELECT x.name,x.amount FROM retail_orders x WHERE x.amount=(SELECT MIN(y.amount) FROM
retail_orders y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Asha | 5200
Dev | 3200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q149. First category above second category maximum — retail orders

```
SELECT name,amount FROM retail_orders WHERE dept='Online' AND amount>(SELECT MAX(amount) FROM
retail_orders WHERE dept='Store')
```

Expected output:

```
Esha | 9200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q150. First category below second category maximum — retail orders

```
SELECT name,amount FROM retail_orders WHERE dept='Online' AND amount<(SELECT MAX(amount) FROM
retail_orders WHERE dept='Store')
```

Expected output:

```
Asha | 5200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q151. Manager names using self join — retail orders

```
SELECT e.name AS member,m.name AS manager FROM retail_orders e LEFT JOIN retail_orders m ON
m.id=e.manager_id ORDER BY e.id
```

Expected output:

```
Asha | NULL
Bala | Asha
Chitra | Asha
Dev | Bala
Esha | Bala
Farah | Chitra
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q152. Direct reports per manager — retail orders

```
SELECT m.name,COUNT(e.id) AS reports FROM retail_orders m JOIN retail_orders e ON
e.manager_id=m.id GROUP BY m.id,m.name ORDER BY m.id
```

Expected output:

```
Asha | 2
Bala | 2
Chitra | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q153. Managers in same city — retail orders

```
SELECT e.name,m.name AS manager FROM retail_orders e JOIN retail_orders m ON m.id=e.manager_id
AND m.city=e.city ORDER BY e.name
```

Expected output:

```
Chitra | Asha
Esha | Bala
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q154. No direct reports using anti join — retail orders

```
SELECT m.name FROM retail_orders m LEFT JOIN retail_orders e ON e.manager_id=m.id WHERE e.id
IS NULL ORDER BY m.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q155. Unique peer pairs — retail orders

```
SELECT a.name,b.name AS peer FROM retail_orders a JOIN retail_orders b ON a.dept=b.dept AND
a.id<b.id ORDER BY a.name,b.name
```

Expected output:

```
Asha | Bala
Asha | Esha
Bala | Esha
Chitra | Dev
Chitra | Farah
Dev | Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q156. Rank by amount — retail orders

```
SELECT name,amount,RANK() OVER(ORDER BY amount DESC) AS rnk FROM retail_orders ORDER BY rnk,
name
```

Expected output:

```
Esha | 9200 | 1
Bala | 7200 | 2
Chitra | 7200 | 2
Asha | 5200 | 4
Farah | 4200 | 5
Dev | 3200 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q157. Dense rank by category — retail orders

```
SELECT name,dept,DENSE_RANK() OVER(PARTITION BY dept ORDER BY amount DESC) AS rnk FROM
retail_orders ORDER BY dept,rnk,name
```

Expected output:

```
Esha | Online | 1
Bala | Online | 2
Asha | Online | 3
Chitra | Store | 1
Farah | Store | 2
Dev | Store | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q158. Top two per category — retail orders

```
SELECT name,dept,amount FROM (SELECT name,dept,amount,ROW_NUMBER() OVER(PARTITION BY dept
ORDER BY amount DESC,id) rn FROM retail_orders) WHERE rn<=2 ORDER BY dept,amount DESC
```

Expected output:

```
Esha | Online | 9200
Bala | Online | 7200
Chitra | Store | 7200
Farah | Store | 4200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q159. Running total — retail orders

```
SELECT name,amount,SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS running_total FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 5200 | 5200
Bala | 7200 | 12400
Chitra | 7200 | 19600
Dev | 3200 | 22800
Esha | 9200 | 32000
Farah | 4200 | 36200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q160. Percentage of total — retail orders

```
SELECT name,ROUND(100.0*amount/SUM(amount) OVER(),2) AS pct FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 14.36
Bala | 19.89
Chitra | 19.89
Dev | 8.84
Esha | 25.41
Farah | 11.6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q161. Previous amount with LAG — retail orders

```
SELECT name,amount,LAG(amount) OVER(ORDER BY id) AS previous FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 5200 | NULL
Bala | 7200 | 5200
Chitra | 7200 | 7200
Dev | 3200 | 7200
Esha | 9200 | 3200
```

```
Farah | 4200 | 9200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q162. Next amount with LEAD — retail orders

```
SELECT name,amount,LEAD(amount) OVER(ORDER BY id) AS next_amount FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 5200 | 7200
Bala | 7200 | 7200
Chitra | 7200 | 3200
Dev | 3200 | 9200
Esha | 9200 | 4200
Farah | 4200 | NULL
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q163. Category total on every row — retail orders

```
SELECT name,dept,SUM(amount) OVER(PARTITION BY dept) AS dept_total FROM retail_orders ORDER BY
id
```

Expected output:

```
Asha | Online | 21600
Bala | Online | 21600
Chitra | Store | 14600
Dev | Store | 14600
Esha | Online | 21600
Farah | Store | 14600
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q164. Row number by score — retail orders

```
SELECT name,score,ROW_NUMBER() OVER(ORDER BY score DESC,id) AS position FROM retail_orders
ORDER BY position
```

Expected output:

```
Esha | 94 | 1
Bala | 91 | 2
Farah | 88 | 3
Asha | 82 | 4
Chitra | 76 | 5
Dev | 65 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q165. Difference from category average — retail orders

```
SELECT name,ROUND(amount-AVG(amount) OVER(PARTITION BY dept),2) AS delta FROM retail_orders
ORDER BY id
```

Expected output:

```
Asha | -2000.0
Bala | 0.0
Chitra | 2333.33
Dev | -1666.67
Esha | 2000.0
Farah | -666.67
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q166. Uppercase names — retail orders

```
SELECT name,UPPER(name) AS normalized FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | ASHA
Bala | BALA
Chitra | CHITRA
Dev | DEV
Esha | ESHA
Farah | FARAH
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q167. First three name characters — retail orders

```
SELECT name,SUBSTR(name,1,3) AS prefix FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | Ash
Bala | Bal
Chitra | Chi
Dev | Dev
Esha | Esh
Farah | Far
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q168. Name length — retail orders

```
SELECT name,LENGTH(name) AS chars FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 4
Bala | 4
Chitra | 6
Dev | 3
Esha | 4
Farah | 5
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q169. Mask lowercase a — retail orders

```
SELECT name,REPLACE(name,'a','*') AS masked FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | Ash*
Bala | B*l*
Chitra | Chitr*
Dev | Dev
Esha | Esh*
Farah | F*r*h
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q170. Amount classification using CASE — retail orders

```
SELECT name,CASE WHEN amount>=8000 THEN 'HIGH' WHEN amount>=5000 THEN 'MEDIUM' ELSE 'LOW' END
AS band FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | MEDIUM
Bala | MEDIUM
Chitra | MEDIUM
Dev | LOW
Esha | HIGH
Farah | LOW
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q171. Replace missing manager with zero — retail orders

```
SELECT name,COALESCE(manager_id,0) AS manager_key FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 0
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q172. Absolute deviation from 6000 — retail orders

```
SELECT name,ABS(amount-6000) AS deviation FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 800
Bala | 1200
Chitra | 1200
Dev | 2800
Esha | 3200
Farah | 1800
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q173. Score remainder — retail orders

```
SELECT name,MOD(score,10) AS remainder FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | 2
Bala | 1
Chitra | 6
Dev | 5
Esha | 4
Farah | 8
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q174. Concatenate category and city — retail orders

```
SELECT name,dept||' / '||city AS label FROM retail_orders ORDER BY id
```

Expected output:

```
Asha | Online / Bengaluru
Bala | Online / Hyderabad
Chitra | Store / Bengaluru
Dev | Store / Chennai
Esha | Online / Hyderabad
```

```
Farah | Store / Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q175. Average score rounded — retail orders

```
SELECT dept,ROUND(AVG(score),1) AS avg_score FROM retail_orders GROUP BY dept ORDER BY dept
```

Expected output:

```
Online | 89.0
Store | 76.3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q176. CTE filtering category totals — retail orders

```
WITH totals AS(SELECT dept,SUM(amount) total FROM retail_orders GROUP BY dept) SELECT dept,
total FROM totals WHERE total>15000 ORDER BY dept
```

Expected output:

```
Online | 21600
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q177. CTE for high scores — retail orders

```
WITH qualified AS(SELECT name,score FROM retail_orders WHERE score>=85) SELECT name,score FROM
qualified ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q178. CTE for highest per city — retail orders

```
WITH ranked AS(SELECT name,city,amount,ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount
DESC,id) rn FROM retail_orders) SELECT name,city,amount FROM ranked WHERE rn=1 ORDER BY city
```

Expected output:

```
Chitra | Bengaluru | 7200
Farah | Chennai | 4200
Esha | Hyderabad | 9200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q179. UNION active and pending cities — retail orders

```
SELECT city FROM retail_orders WHERE status='ACTIVE' UNION SELECT city FROM retail_orders
WHERE status='PENDING' ORDER BY city
```

Expected output:

```
Bengaluru
Chennai
Hyderabad
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q180. INTERSECT cities across categories — retail orders

```
SELECT city FROM retail_orders WHERE dept='Online' INTERSECT SELECT city FROM retail_orders
WHERE dept='Store' ORDER BY city
```

Expected output:

```
Bengaluru
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Business case 4: Insurance Claims

Q181. Active values above 5000 — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE status='ACTIVE' AND amount>5000 ORDER BY id
```

Expected output:

```
Asha | 5300
Bala | 7300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q182. Values within a range — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE amount BETWEEN 4000 AND 7000 ORDER BY amount,
name
```

Expected output:

```
Farah | 4300
Asha | 5300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q183. Cities using IN — insurance claims

```
SELECT name,city FROM insurance_claims WHERE city IN ('Chennai','Hyderabad') ORDER BY name
```

Expected output:

```
Bala | Hyderabad
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q184. Names starting with A — insurance claims

```
SELECT name FROM insurance_claims WHERE name LIKE 'A%'
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q185. Records with missing managers — insurance claims

```
SELECT name FROM insurance_claims WHERE manager_id IS NULL
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q186. Records with valid manager IDs — insurance claims

```
SELECT name,manager_id FROM insurance_claims WHERE manager_id IS NOT NULL ORDER BY id
```

Expected output:

```
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q187. Exclude closed records — insurance claims

```
SELECT name,status FROM insurance_claims WHERE status<>'CLOSED' ORDER BY id
```

Expected output:

```
Asha | ACTIVE
Bala | ACTIVE
Chitra | PENDING
Dev | ACTIVE
Farah | ACTIVE
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q188. First category only — insurance claims

```
SELECT name FROM insurance_claims WHERE dept='Health' ORDER BY name
```

Expected output:

```
Asha
Bala
Esha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q189. Descending values and stable ties — insurance claims

```
SELECT name,amount FROM insurance_claims ORDER BY amount DESC,name
```

Expected output:

```
Esha | 9300
Bala | 7300
Chitra | 7300
Asha | 5300
Farah | 4300
Dev | 3300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q190. Scores above 85 — insurance claims

```
SELECT name,score FROM insurance_claims WHERE score>85 ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q191. Count records by status — insurance claims

```
SELECT status,COUNT(*) AS n FROM insurance_claims GROUP BY status ORDER BY status
```

Expected output:

```
ACTIVE | 4
CLOSED | 1
PENDING | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q192. Sum amount by category — insurance claims

```
SELECT dept,SUM(amount) AS total FROM insurance_claims GROUP BY dept ORDER BY dept
```

Expected output:

```
Health | 21900
Motor | 14900
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q193. Average amount by city — insurance claims

```
SELECT city,ROUND(AVG(amount),2) AS avg_amount FROM insurance_claims GROUP BY city ORDER BY
city
```

Expected output:

```
Bengaluru | 6300.0
Chennai | 3800.0
Hyderabad | 8300.0
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q194. Maximum amount by category — insurance claims

```
SELECT dept,MAX(amount) AS maximum FROM insurance_claims GROUP BY dept ORDER BY dept
```

Expected output:

```
Health | 9300
Motor | 7300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q195. Minimum amount by city — insurance claims

```
SELECT city,MIN(amount) AS minimum FROM insurance_claims GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 5300
Chennai | 3300
Hyderabad | 7300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q196. Distinct active cities — insurance claims

```
SELECT COUNT(DISTINCT city) AS cities FROM insurance_claims WHERE status='ACTIVE'
```

Expected output:

```
3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q197. Categories with three records — insurance claims

```
SELECT dept,COUNT(*) AS n FROM insurance_claims GROUP BY dept HAVING COUNT(*)>=3 ORDER BY dept
```

Expected output:

```
Health | 3
Motor | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q198. Conditional active sum — insurance claims

```
SELECT SUM(CASE WHEN status='ACTIVE' THEN amount ELSE 0 END) AS active_total FROM
insurance_claims
```

Expected output:

```
20200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q199. Cities having multiple records — insurance claims

```
SELECT city,COUNT(*) AS n FROM insurance_claims GROUP BY city HAVING COUNT(*)>1 ORDER BY city
```

Expected output:

```
Bengaluru | 2
Chennai | 2
Hyderabad | 2
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q200. Grand total and average — insurance claims

```
SELECT SUM(amount) AS total,ROUND(AVG(amount),2) AS average FROM insurance_claims
```

Expected output:

```
36800 | 6133.33
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q201. Above overall average — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE amount>(SELECT AVG(amount) FROM
insurance_claims) ORDER BY amount DESC
```

Expected output:

```
Esha | 9300
Bala | 7300
Chitra | 7300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q202. Highest amount including ties — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE amount=(SELECT MAX(amount) FROM
insurance_claims)
```

Expected output:

```
Esha | 9300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q203. Second distinct highest amount — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE amount=(SELECT MAX(amount) FROM
insurance_claims WHERE amount<(SELECT MAX(amount) FROM insurance_claims)) ORDER BY name
```

Expected output:

```
Bala | 7300
Chitra | 7300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q204. Above own category average — insurance claims

```
SELECT x.name,x.amount FROM insurance_claims x WHERE x.amount>(SELECT AVG(y.amount) FROM
insurance_claims y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Chitra | 7300
Esha | 9300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q205. Records from repeated cities — insurance claims

```
SELECT name,city FROM insurance_claims WHERE city IN (SELECT city FROM insurance_claims GROUP
BY city HAVING COUNT(*)>1) ORDER BY name
```

Expected output:

```
Asha | Bengaluru
Bala | Hyderabad
Chitra | Bengaluru
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q206. Records with existing manager — insurance claims

```
SELECT x.name FROM insurance_claims x WHERE EXISTS(SELECT 1 FROM insurance_claims m WHERE
m.id=x.manager_id) ORDER BY x.name
```

Expected output:

```
Bala
Chitra
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q207. Records managing nobody — insurance claims

```
SELECT x.name FROM insurance_claims x WHERE NOT EXISTS(SELECT 1 FROM insurance_claims y WHERE
y.manager_id=x.id) ORDER BY x.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q208. Category-wise minimum records — insurance claims

```
SELECT x.name,x.amount FROM insurance_claims x WHERE x.amount=(SELECT MIN(y.amount) FROM
insurance_claims y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Asha | 5300
Dev | 3300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q209. First category above second category maximum — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE dept='Health' AND amount>(SELECT MAX(amount)
FROM insurance_claims WHERE dept='Motor')
```

Expected output:

```
Esha | 9300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q210. First category below second category maximum — insurance claims

```
SELECT name,amount FROM insurance_claims WHERE dept='Health' AND amount<(SELECT MAX(amount)
FROM insurance_claims WHERE dept='Motor')
```

Expected output:

```
Asha | 5300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q211. Manager names using self join — insurance claims

```
SELECT e.name AS member,m.name AS manager FROM insurance_claims e LEFT JOIN insurance_claims m
ON m.id=e.manager_id ORDER BY e.id
```

Expected output:

```
Asha | NULL
Bala | Asha
Chitra | Asha
Dev | Bala
Esha | Bala
Farah | Chitra
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q212. Direct reports per manager — insurance claims

```
SELECT m.name,COUNT(e.id) AS reports FROM insurance_claims m JOIN insurance_claims e ON
e.manager_id=m.id GROUP BY m.id,m.name ORDER BY m.id
```

Expected output:

```
Asha | 2
Bala | 2
```

```
Chitra | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q213. Managers in same city — insurance claims

```
SELECT e.name,m.name AS manager FROM insurance_claims e JOIN insurance_claims m ON
m.id=e.manager_id AND m.city=e.city ORDER BY e.name
```

Expected output:

```
Chitra | Asha
Esha | Bala
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q214. No direct reports using anti join — insurance claims

```
SELECT m.name FROM insurance_claims m LEFT JOIN insurance_claims e ON e.manager_id=m.id WHERE
e.id IS NULL ORDER BY m.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q215. Unique peer pairs — insurance claims

```
SELECT a.name,b.name AS peer FROM insurance_claims a JOIN insurance_claims b ON a.dept=b.dept
AND a.id<b.id ORDER BY a.name,b.name
```

Expected output:

```
Asha | Bala
Asha | Esha
Bala | Esha
Chitra | Dev
Chitra | Farah
Dev | Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q216. Rank by amount — insurance claims

```
SELECT name,amount,RANK() OVER(ORDER BY amount DESC) AS rnk FROM insurance_claims ORDER BY rnk,
name
```

Expected output:

```
Esha | 9300 | 1
Bala | 7300 | 2
Chitra | 7300 | 2
Asha | 5300 | 4
Farah | 4300 | 5
Dev | 3300 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q217. Dense rank by category — insurance claims

```
SELECT name,dept,DENSE_RANK() OVER(PARTITION BY dept ORDER BY amount DESC) AS rnk FROM
```

```
insurance_claims ORDER BY dept,rnk,name
```

Expected output:

```
Esha | Health | 1
Bala | Health | 2
Asha | Health | 3
Chitra | Motor | 1
Farah | Motor | 2
Dev | Motor | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q218. Top two per category — insurance claims

```
SELECT name,dept,amount FROM (SELECT name,dept,amount,ROW_NUMBER() OVER(PARTITION BY dept
ORDER BY amount DESC,id) rn FROM insurance_claims) WHERE rn<=2 ORDER BY dept,amount DESC
```

Expected output:

```
Esha | Health | 9300
Bala | Health | 7300
Chitra | Motor | 7300
Farah | Motor | 4300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q219. Running total — insurance claims

```
SELECT name,amount,SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS running_total FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 5300 | 5300
Bala | 7300 | 12600
Chitra | 7300 | 19900
Dev | 3300 | 23200
Esha | 9300 | 32500
Farah | 4300 | 36800
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q220. Percentage of total — insurance claims

```
SELECT name,ROUND(100.0*amount/SUM(amount) OVER(),2) AS pct FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 14.4
Bala | 19.84
Chitra | 19.84
Dev | 8.97
Esha | 25.27
Farah | 11.68
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q221. Previous amount with LAG — insurance claims

```
SELECT name,amount,LAG(amount) OVER(ORDER BY id) AS previous FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 5300 | NULL
Bala | 7300 | 5300
Chitra | 7300 | 7300
Dev | 3300 | 7300
```

```
Esha | 9300 | 3300
Farah | 4300 | 9300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q222. Next amount with LEAD — insurance claims

```
SELECT name,amount,LEAD(amount) OVER(ORDER BY id) AS next_amount FROM insurance_claims ORDER
BY id
```

Expected output:

```
Asha | 5300 | 7300
Bala | 7300 | 7300
Chitra | 7300 | 3300
Dev | 3300 | 9300
Esha | 9300 | 4300
Farah | 4300 | NULL
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q223. Category total on every row — insurance claims

```
SELECT name,dept,SUM(amount) OVER(PARTITION BY dept) AS dept_total FROM insurance_claims ORDER
BY id
```

Expected output:

```
Asha | Health | 21900
Bala | Health | 21900
Chitra | Motor | 14900
Dev | Motor | 14900
Esha | Health | 21900
Farah | Motor | 14900
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q224. Row number by score — insurance claims

```
SELECT name,score,ROW_NUMBER() OVER(ORDER BY score DESC,id) AS position FROM insurance_claims
ORDER BY position
```

Expected output:

```
Esha | 94 | 1
Bala | 91 | 2
Farah | 88 | 3
Asha | 82 | 4
Chitra | 76 | 5
Dev | 65 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q225. Difference from category average — insurance claims

```
SELECT name,ROUND(amount-AVG(amount) OVER(PARTITION BY dept),2) AS delta FROM insurance_claims
ORDER BY id
```

Expected output:

```
Asha | -2000.0
Bala | 0.0
Chitra | 2333.33
Dev | -1666.67
Esha | 2000.0
Farah | -666.67
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q226. Uppercase names — insurance claims

```
SELECT name,UPPER(name) AS normalized FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | ASHA
Bala | BALA
Chitra | CHITRA
Dev | DEV
Esha | ESHA
Farah | FARAH
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q227. First three name characters — insurance claims

```
SELECT name,SUBSTR(name,1,3) AS prefix FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | Ash
Bala | Bal
Chitra | Chi
Dev | Dev
Esha | Esh
Farah | Far
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q228. Name length — insurance claims

```
SELECT name,LENGTH(name) AS chars FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 4
Bala | 4
Chitra | 6
Dev | 3
Esha | 4
Farah | 5
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q229. Mask lowercase a — insurance claims

```
SELECT name,REPLACE(name,'a','*') AS masked FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | Ash*
Bala | B*l*
Chitra | Chitr*
Dev | Dev
Esha | Esh*
Farah | F*r*h
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q230. Amount classification using CASE — insurance claims

```
SELECT name,CASE WHEN amount>=8000 THEN 'HIGH' WHEN amount>=5000 THEN 'MEDIUM' ELSE 'LOW' END
AS band FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | MEDIUM
Bala | MEDIUM
Chitra | MEDIUM
Dev | LOW
Esha | HIGH
Farah | LOW
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q231. Replace missing manager with zero — insurance claims

```
SELECT name,COALESCE(manager_id,0) AS manager_key FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 0
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q232. Absolute deviation from 6000 — insurance claims

```
SELECT name,ABS(amount-6000) AS deviation FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 700
Bala | 1300
Chitra | 1300
Dev | 2700
Esha | 3300
Farah | 1700
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q233. Score remainder — insurance claims

```
SELECT name,MOD(score,10) AS remainder FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | 2
Bala | 1
Chitra | 6
Dev | 5
Esha | 4
Farah | 8
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q234. Concatenate category and city — insurance claims

```
SELECT name,dept||' / '||city AS label FROM insurance_claims ORDER BY id
```

Expected output:

```
Asha | Health / Bengaluru
Bala | Health / Hyderabad
Chitra | Motor / Bengaluru
Dev | Motor / Chennai
Esha | Health / Hyderabad
Farah | Motor / Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q235. Average score rounded — insurance claims

```
SELECT dept,ROUND(AVG(score),1) AS avg_score FROM insurance_claims GROUP BY dept ORDER BY dept
```

Expected output:

```
Health | 89.0
Motor | 76.3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q236. CTE filtering category totals — insurance claims

```
WITH totals AS(SELECT dept,SUM(amount) total FROM insurance_claims GROUP BY dept) SELECT dept,
total FROM totals WHERE total>15000 ORDER BY dept
```

Expected output:

```
Health | 21900
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q237. CTE for high scores — insurance claims

```
WITH qualified AS(SELECT name,score FROM insurance_claims WHERE score>=85) SELECT name,score
FROM qualified ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q238. CTE for highest per city — insurance claims

```
WITH ranked AS(SELECT name,city,amount,ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount
DESC,id) rn FROM insurance_claims) SELECT name,city,amount FROM ranked WHERE rn=1 ORDER BY city
```

Expected output:

```
Chitra | Bengaluru | 7300
Farah | Chennai | 4300
Esha | Hyderabad | 9300
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q239. UNION active and pending cities — insurance claims

```
SELECT city FROM insurance_claims WHERE status='ACTIVE' UNION SELECT city FROM
insurance_claims WHERE status='PENDING' ORDER BY city
```

Expected output:

```
Bengaluru
```

```
Chennai
Hyderabad
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q240. INTERSECT cities across categories — insurance claims

```
SELECT city FROM insurance_claims WHERE dept='Health' INTERSECT SELECT city FROM
insurance_claims WHERE dept='Motor' ORDER BY city
```

Expected output:

```
Bengaluru
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Business case 5: Student Fees

Q241. Active values above 5000 — student fees

```
SELECT name,amount FROM student_fees WHERE status='ACTIVE' AND amount>5000 ORDER BY id
```

Expected output:

```
Asha | 5400
Bala | 7400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q242. Values within a range — student fees

```
SELECT name,amount FROM student_fees WHERE amount BETWEEN 4000 AND 7000 ORDER BY amount,name
```

Expected output:

```
Farah | 4400
Asha | 5400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q243. Cities using IN — student fees

```
SELECT name,city FROM student_fees WHERE city IN ('Chennai','Hyderabad') ORDER BY name
```

Expected output:

```
Bala | Hyderabad
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q244. Names starting with A — student fees

```
SELECT name FROM student_fees WHERE name LIKE 'A%'
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q245. Records with missing managers — student fees

```
SELECT name FROM student_fees WHERE manager_id IS NULL
```

Expected output:

```
Asha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q246. Records with valid manager IDs — student fees

```
SELECT name,manager_id FROM student_fees WHERE manager_id IS NOT NULL ORDER BY id
```

Expected output:

```
Bala | 1
```

```
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q247. Exclude closed records — student fees

```
SELECT name,status FROM student_fees WHERE status<>'CLOSED' ORDER BY id
```

Expected output:

```
Asha | ACTIVE
Bala | ACTIVE
Chitra | PENDING
Dev | ACTIVE
Farah | ACTIVE
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q248. First category only — student fees

```
SELECT name FROM student_fees WHERE dept='Data Engineering' ORDER BY name
```

Expected output:

```
Asha
Bala
Esha
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q249. Descending values and stable ties — student fees

```
SELECT name,amount FROM student_fees ORDER BY amount DESC,name
```

Expected output:

```
Esha | 9400
Bala | 7400
Chitra | 7400
Asha | 5400
Farah | 4400
Dev | 3400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q250. Scores above 85 — student fees

```
SELECT name,score FROM student_fees WHERE score>85 ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q251. Count records by status — student fees

```
SELECT status,COUNT(*) AS n FROM student_fees GROUP BY status ORDER BY status
```

Expected output:

```
ACTIVE | 4
CLOSED | 1
PENDING | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q252. Sum amount by category — student fees

```
SELECT dept,SUM(amount) AS total FROM student_fees GROUP BY dept ORDER BY dept
```

Expected output:

```
Data Engineering | 22200
DevOps | 15200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q253. Average amount by city — student fees

```
SELECT city,ROUND(AVG(amount),2) AS avg_amount FROM student_fees GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 6400.0
Chennai | 3900.0
Hyderabad | 8400.0
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q254. Maximum amount by category — student fees

```
SELECT dept,MAX(amount) AS maximum FROM student_fees GROUP BY dept ORDER BY dept
```

Expected output:

```
Data Engineering | 9400
DevOps | 7400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q255. Minimum amount by city — student fees

```
SELECT city,MIN(amount) AS minimum FROM student_fees GROUP BY city ORDER BY city
```

Expected output:

```
Bengaluru | 5400
Chennai | 3400
Hyderabad | 7400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q256. Distinct active cities — student fees

```
SELECT COUNT(DISTINCT city) AS cities FROM student_fees WHERE status='ACTIVE'
```

Expected output:

```
3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q257. Categories with three records — student fees

```
SELECT dept,COUNT(*) AS n FROM student_fees GROUP BY dept HAVING COUNT(*)>=3 ORDER BY dept
```

Expected output:

```
Data Engineering | 3
DevOps | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q258. Conditional active sum — student fees

```
SELECT SUM(CASE WHEN status='ACTIVE' THEN amount ELSE 0 END) AS active_total FROM student_fees
```

Expected output:

```
20600
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q259. Cities having multiple records — student fees

```
SELECT city,COUNT(*) AS n FROM student_fees GROUP BY city HAVING COUNT(*)>1 ORDER BY city
```

Expected output:

```
Bengaluru | 2
Chennai | 2
Hyderabad | 2
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q260. Grand total and average — student fees

```
SELECT SUM(amount) AS total,ROUND(AVG(amount),2) AS average FROM student_fees
```

Expected output:

```
37400 | 6233.33
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q261. Above overall average — student fees

```
SELECT name,amount FROM student_fees WHERE amount>(SELECT AVG(amount) FROM student_fees) ORDER
BY amount DESC
```

Expected output:

```
Esha | 9400
Bala | 7400
Chitra | 7400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q262. Highest amount including ties — student fees

```
SELECT name,amount FROM student_fees WHERE amount=(SELECT MAX(amount) FROM student_fees)
```

Expected output:

```
Esha | 9400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q263. Second distinct highest amount — student fees

```
SELECT name,amount FROM student_fees WHERE amount=(SELECT MAX(amount) FROM student_fees WHERE
amount<(SELECT MAX(amount) FROM student_fees)) ORDER BY name
```

Expected output:

```
Bala | 7400
Chitra | 7400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q264. Above own category average — student fees

```
SELECT x.name,x.amount FROM student_fees x WHERE x.amount>(SELECT AVG(y.amount) FROM
student_fees y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Chitra | 7400
Esha | 9400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q265. Records from repeated cities — student fees

```
SELECT name,city FROM student_fees WHERE city IN (SELECT city FROM student_fees GROUP BY city
HAVING COUNT(*)>1) ORDER BY name
```

Expected output:

```
Asha | Bengaluru
Bala | Hyderabad
Chitra | Bengaluru
Dev | Chennai
Esha | Hyderabad
Farah | Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q266. Records with existing manager — student fees

```
SELECT x.name FROM student_fees x WHERE EXISTS(SELECT 1 FROM student_fees m WHERE
m.id=x.manager_id) ORDER BY x.name
```

Expected output:

```
Bala
Chitra
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q267. Records managing nobody — student fees

```
SELECT x.name FROM student_fees x WHERE NOT EXISTS(SELECT 1 FROM student_fees y WHERE
y.manager_id=x.id) ORDER BY x.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q268. Category-wise minimum records — student fees

```
SELECT x.name,x.amount FROM student_fees x WHERE x.amount=(SELECT MIN(y.amount) FROM
student_fees y WHERE y.dept=x.dept) ORDER BY x.name
```

Expected output:

```
Asha | 5400
Dev | 3400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q269. First category above second category maximum — student fees

```
SELECT name,amount FROM student_fees WHERE dept='Data Engineering' AND amount>(SELECT
MAX(amount) FROM student_fees WHERE dept='DevOps')
```

Expected output:

```
Esha | 9400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q270. First category below second category maximum — student fees

```
SELECT name,amount FROM student_fees WHERE dept='Data Engineering' AND amount<(SELECT
MAX(amount) FROM student_fees WHERE dept='DevOps')
```

Expected output:

```
Asha | 5400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q271. Manager names using self join — student fees

```
SELECT e.name AS member,m.name AS manager FROM student_fees e LEFT JOIN student_fees m ON
m.id=e.manager_id ORDER BY e.id
```

Expected output:

```
Asha | NULL
Bala | Asha
Chitra | Asha
Dev | Bala
Esha | Bala
Farah | Chitra
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q272. Direct reports per manager — student fees

```
SELECT m.name,COUNT(e.id) AS reports FROM student_fees m JOIN student_fees e ON
e.manager_id=m.id GROUP BY m.id,m.name ORDER BY m.id
```

Expected output:

```
Asha | 2
Bala | 2
Chitra | 1
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q273. Managers in same city — student fees

```
SELECT e.name,m.name AS manager FROM student_fees e JOIN student_fees m ON m.id=e.manager_id
AND m.city=e.city ORDER BY e.name
```

Expected output:

```
Chitra | Asha
Esha | Bala
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q274. No direct reports using anti join — student fees

```
SELECT m.name FROM student_fees m LEFT JOIN student_fees e ON e.manager_id=m.id WHERE e.id IS
NULL ORDER BY m.name
```

Expected output:

```
Dev
Esha
Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q275. Unique peer pairs — student fees

```
SELECT a.name,b.name AS peer FROM student_fees a JOIN student_fees b ON a.dept=b.dept AND
a.id<b.id ORDER BY a.name,b.name
```

Expected output:

```
Asha | Bala
Asha | Esha
Bala | Esha
Chitra | Dev
Chitra | Farah
Dev | Farah
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q276. Rank by amount — student fees

```
SELECT name,amount,RANK() OVER(ORDER BY amount DESC) AS rnk FROM student_fees ORDER BY rnk,name
```

Expected output:

```
Esha | 9400 | 1
Bala | 7400 | 2
Chitra | 7400 | 2
Asha | 5400 | 4
Farah | 4400 | 5
Dev | 3400 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q277. Dense rank by category — student fees

```
SELECT name,dept,DENSE_RANK() OVER(PARTITION BY dept ORDER BY amount DESC) AS rnk FROM
student_fees ORDER BY dept,rnk,name
```

Expected output:

```
Esha | Data Engineering | 1
Bala | Data Engineering | 2
Asha | Data Engineering | 3
Chitra | DevOps | 1
Farah | DevOps | 2
Dev | DevOps | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q278. Top two per category — student fees

```
SELECT name,dept,amount FROM (SELECT name,dept,amount,ROW_NUMBER() OVER(PARTITION BY dept
ORDER BY amount DESC,id) rn FROM student_fees) WHERE rn<=2 ORDER BY dept,amount DESC
```

Expected output:

```
Esha | Data Engineering | 9400
Bala | Data Engineering | 7400
Chitra | DevOps | 7400
Farah | DevOps | 4400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q279. Running total — student fees

```
SELECT name,amount,SUM(amount) OVER(ORDER BY id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT
ROW) AS running_total FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 5400 | 5400
Bala | 7400 | 12800
Chitra | 7400 | 20200
Dev | 3400 | 23600
Esha | 9400 | 33000
Farah | 4400 | 37400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q280. Percentage of total — student fees

```
SELECT name,ROUND(100.0*amount/SUM(amount) OVER(),2) AS pct FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 14.44
Bala | 19.79
Chitra | 19.79
Dev | 9.09
Esha | 25.13
Farah | 11.76
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q281. Previous amount with LAG — student fees

```
SELECT name,amount,LAG(amount) OVER(ORDER BY id) AS previous FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 5400 | NULL
Bala | 7400 | 5400
Chitra | 7400 | 7400
Dev | 3400 | 7400
Esha | 9400 | 3400
```

```
Farah | 4400 | 9400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q282. Next amount with LEAD — student fees

```
SELECT name,amount,LEAD(amount) OVER(ORDER BY id) AS next_amount FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 5400 | 7400
Bala | 7400 | 7400
Chitra | 7400 | 3400
Dev | 3400 | 9400
Esha | 9400 | 4400
Farah | 4400 | NULL
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q283. Category total on every row — student fees

```
SELECT name,dept,SUM(amount) OVER(PARTITION BY dept) AS dept_total FROM student_fees ORDER BY
id
```

Expected output:

```
Asha | Data Engineering | 22200
Bala | Data Engineering | 22200
Chitra | DevOps | 15200
Dev | DevOps | 15200
Esha | Data Engineering | 22200
Farah | DevOps | 15200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q284. Row number by score — student fees

```
SELECT name,score,ROW_NUMBER() OVER(ORDER BY score DESC,id) AS position FROM student_fees
ORDER BY position
```

Expected output:

```
Esha | 94 | 1
Bala | 91 | 2
Farah | 88 | 3
Asha | 82 | 4
Chitra | 76 | 5
Dev | 65 | 6
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q285. Difference from category average — student fees

```
SELECT name,ROUND(amount-AVG(amount) OVER(PARTITION BY dept),2) AS delta FROM student_fees
ORDER BY id
```

Expected output:

```
Asha | -2000.0
Bala | 0.0
Chitra | 2333.33
Dev | -1666.67
Esha | 2000.0
Farah | -666.67
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q286. Uppercase names — student fees

```
SELECT name,UPPER(name) AS normalized FROM student_fees ORDER BY id
```

Expected output:

```
Asha | ASHA
Bala | BALA
Chitra | CHITRA
Dev | DEV
Esha | ESHA
Farah | FARAH
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q287. First three name characters — student fees

```
SELECT name,SUBSTR(name,1,3) AS prefix FROM student_fees ORDER BY id
```

Expected output:

```
Asha | Ash
Bala | Bal
Chitra | Chi
Dev | Dev
Esha | Esh
Farah | Far
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q288. Name length — student fees

```
SELECT name,LENGTH(name) AS chars FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 4
Bala | 4
Chitra | 6
Dev | 3
Esha | 4
Farah | 5
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q289. Mask lowercase a — student fees

```
SELECT name,REPLACE(name,'a','*') AS masked FROM student_fees ORDER BY id
```

Expected output:

```
Asha | Ash*
Bala | B*l*
Chitra | Chitr*
Dev | Dev
Esha | Esh*
Farah | F*r*h
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q290. Amount classification using CASE — student fees

```
SELECT name,CASE WHEN amount>=8000 THEN 'HIGH' WHEN amount>=5000 THEN 'MEDIUM' ELSE 'LOW' END
AS band FROM student_fees ORDER BY id
```

Expected output:

```
Asha | MEDIUM
Bala | MEDIUM
Chitra | MEDIUM
Dev | LOW
Esha | HIGH
Farah | LOW
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q291. Replace missing manager with zero — student fees

```
SELECT name,COALESCE(manager_id,0) AS manager_key FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 0
Bala | 1
Chitra | 1
Dev | 2
Esha | 2
Farah | 3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q292. Absolute deviation from 6000 — student fees

```
SELECT name,ABS(amount-6000) AS deviation FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 600
Bala | 1400
Chitra | 1400
Dev | 2600
Esha | 3400
Farah | 1600
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q293. Score remainder — student fees

```
SELECT name,MOD(score,10) AS remainder FROM student_fees ORDER BY id
```

Expected output:

```
Asha | 2
Bala | 1
Chitra | 6
Dev | 5
Esha | 4
Farah | 8
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q294. Concatenate category and city — student fees

```
SELECT name,dept||' / '||city AS label FROM student_fees ORDER BY id
```

Expected output:

```
Asha | Data Engineering / Bengaluru
Bala | Data Engineering / Hyderabad
Chitra | DevOps / Bengaluru
Dev | DevOps / Chennai
Esha | Data Engineering / Hyderabad
```

```
Farah | DevOps / Chennai
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q295. Average score rounded — student fees

```
SELECT dept,ROUND(AVG(score),1) AS avg_score FROM student_fees GROUP BY dept ORDER BY dept
```

Expected output:

```
Data Engineering | 89.0
DevOps | 76.3
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q296. CTE filtering category totals — student fees

```
WITH totals AS(SELECT dept,SUM(amount) total FROM student_fees GROUP BY dept) SELECT dept,
total FROM totals WHERE total>15000 ORDER BY dept
```

Expected output:

```
Data Engineering | 22200
DevOps | 15200
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q297. CTE for high scores — student fees

```
WITH qualified AS(SELECT name,score FROM student_fees WHERE score>=85) SELECT name,score FROM
qualified ORDER BY score DESC
```

Expected output:

```
Esha | 94
Bala | 91
Farah | 88
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q298. CTE for highest per city — student fees

```
WITH ranked AS(SELECT name,city,amount,ROW_NUMBER() OVER(PARTITION BY city ORDER BY amount
DESC,id) rn FROM student_fees) SELECT name,city,amount FROM ranked WHERE rn=1 ORDER BY city
```

Expected output:

```
Chitra | Bengaluru | 7400
Farah | Chennai | 4400
Esha | Hyderabad | 9400
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q299. UNION active and pending cities — student fees

```
SELECT city FROM student_fees WHERE status='ACTIVE' UNION SELECT city FROM student_fees WHERE
status='PENDING' ORDER BY city
```

Expected output:

```
Bengaluru
Chennai
Hyderabad
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.

Q300. INTERSECT cities across categories — student fees

```
SELECT city FROM student_fees WHERE dept='Data Engineering' INTERSECT SELECT city FROM
student_fees WHERE dept='DevOps' ORDER BY city
```

Expected output:

```
Bengaluru
```

Interview reasoning: Explain the filtering, grouping, join condition or window frame; discuss NULLs, ties and deterministic ordering where applicable.